

Let's Build GPT: From Scratch, in Code, Spelled Out

Karpathy, A. (2023). "Let's build GPT: from scratch, in code, spelled out." YouTube — Neural Networks: Zero to Hero.
Published 17 January 2023.

ASSIGNED READING

Video — watch the first ~30 min (data → tokenisation → the self-attention idea)

~30 min

Optional — the full lecture builds a working GPT (nanoGPT) end-to-end

~1 h 56 m

» ~30 min video • this summary + reasoning notes ~20 min

Note on currency. The lecture is from January 2023 and builds a tiny character-level GPT (Generative Pre-trained Transformer). The model is small, but the mechanism — attention inside a transformer, predicting the next token — is exactly what today's frontier models still do. Watch it for the mechanism, not the model size. You do not need to code along.

KEY TAKEAWAY

A GPT is a next-token predictor. Karpathy builds one from nothing: he trains it on a small text file, and the whole model does just one thing — given a sequence of tokens, output a probability distribution over what the next token should be. Everything impressive (and everything wrong) about large language models grows from repeating that single operation. The engine that makes it work is self-attention: each token looks back at earlier tokens and pulls in the information it needs.

WHY IT MATTERS FOR THIS COURSE

This is Capsule C1 ("Inside the model") made concrete. In Week 2 you rented a foundation model as a black box; here you watch someone open it and wire up the parts — tokens, an attention block, a transformer, training. You don't need to follow the code line by line. What you need is the intuition: why the model predicts the next token, what attention actually computes, and why the same mechanism produces both fluency and hallucination. That intuition is the ground the prompting capsule (C2) stands on — prompting only makes sense once you know what you are steering.

Feeds Class 3: C1 (tokens • attention • transformer • why it hallucinates) • sets up C2 (why prompting works).

MAIN POINTS

The task: predict the next token

- **Characters in, probabilities out.** The demo reads text one token at a time (here a token is a single character) and learns to predict the next one. Sampling from that prediction over and over is how it "writes" — after training on a small Shakespeare file it babbles Shakespeare-like text.
- **Tokenisation.** Before a model sees any text, the text is chopped into **tokens**; each token is mapped to an integer, and each integer to a vector of numbers. Karpathy uses single characters as tokens to keep the demo simple; production models use **sub-word** tokens — a common word is usually one token, a rare word is split into fragments. The model never sees your sentence. It sees a list of integers.
- **Tokenisation, worked.** Take "Unbelievably, strawberry costs \$1,000,000." A typical sub-word tokeniser turns that into roughly **Un | bel | iev | ably | , | straw | berry | costs | \$ | 1 | ,000 | ,000 | .** — thirteen tokens, and not one of them is a letter. So when you ask "how many r's are in **strawberry**?" you are asking about something the model cannot see: it holds two opaque chunks, **straw** and **berry**, not ten letters. It answers from what training text **says about** the word, not by counting. The same goes for arithmetic on **1 | ,000 | ,000** — digit groups, not a number. (Callback to the Class 1 demo.)
- **Tokens are also the bill and the budget.** Providers price per token and cap the context window in tokens, so tokenisation is a commercial fact, not just a technical one. English runs at roughly 0.75 words per token. Chinese, Japanese, Korean and Indic scripts have historically tokenised far less efficiently — the same content costing two to three times the tokens; newer tokenisers have narrowed that gap but not closed it. Practical effect: the same document can cost noticeably more, and fill the window faster, in one language than another.

Start dumb: the bigram baseline

- **A model with no context.** He first builds a bigram model that predicts the next token from only the current one — no memory of what came before. It works but is hopeless, which motivates the real question: how can a token take account of the tokens before it?

Self-attention — the core idea

- **Every token gathers from the past.** Attention lets each position build a weighted average of information from earlier positions. Weights are high where an earlier token is relevant to the current one, low elsewhere — so context flows forward.
- **Query, Key, Value (QKV).** Attention needs some way to decide **which** earlier tokens matter. So each token projects its own vector into three smaller ones: a **query** (“what am I looking for?”), a **key** (“what do I contain — what am I useful for?”) and a **value** (“what I hand over if someone attends to me”). The current token’s query is compared against every earlier token’s key by dot product — high match, high weight. The weights are scaled, passed through a softmax so they sum to one, and used to take a weighted average of the **values**. That average is what the token carries forward. All of it is matrix multiplication, which is why it runs fast on a graphics processing unit (GPU).
- **QKV, worked.** “The nurse told the patient that she would be back in an hour.” When the model reaches **she**, that token’s query is roughly “I am a singular pronoun — who do I refer to?” Every earlier token offers a key: **nurse** advertises “person, subject, the one doing the telling”; **patient** advertises “person, object, the one being told”; **hour** advertises “span of time.” The query matches **nurse**’s key most strongly, so most of the weight lands there and **nurse**’s value — its meaning — flows into the representation of **she**. Change the sentence to “...that she would be discharged in an hour” and the same machinery shifts weight toward **patient**. Nothing here is hard-coded: the three projections that produce Q, K and V are learned from text.
- **The library analogy.** The query is the question you ask at the desk. The key is the label on each book’s spine. The value is what is actually inside the book. Ordinary search hands you the one best-matching book; attention hands you a **blend** of every book on the shelf, mixed in proportion to how well each spine matched. That “soft” lookup is what makes attention differentiable — and therefore learnable by gradient descent.
- **Causal masking.** For text generation a token may look only backwards, never at the future — otherwise it would cheat by seeing the answer. This “causal mask” is what makes the model a generator.

From attention to a Transformer

- **Multi-head — why one head is not enough.** A single attention head produces one weighted average per token, which means it can express **one** relationship at a time. But a word usually stands in several relationships at once. In “The nurse told the patient that she would be back,” the token **she** needs, simultaneously: who it refers to; that its verb is “would be”; and where it sits in the clause structure. With one head, the model has to choose.
- **What multi-head achieves.** So the vector is split into **h** slices and **h** heads run in parallel, each with its own learned Q, K and V projections; their outputs are concatenated and mixed by one further learned matrix. Interpretability work finds heads that appear to specialise — one tracking pronoun reference, another subject-verb agreement, another simple position. Three practical consequences: (i) many relationships get captured within a single layer instead of needing many layers; (ii) the heads overlap enough that the model degrades gracefully when one is unhelpful, rather than failing outright; (iii) because each head is narrower, running **h** of them costs about the same as one wide head — and they run in parallel, so there is no extra wall-clock time. GPT-2 small used 12 heads per layer; frontier models use dozens.
- **Then stack.** One block = multi-head attention + a small feed-forward network, with residual connections and normalisation around them. Stack that block many times, add positional information so word order matters, and you have the Transformer of the 2017 paper “Attention Is All You Need” — the architecture of Karpathy’s nanoGPT and, scaled up, of every frontier model you have used.
- **Where this architecture came from (→ Reading 2).** The Transformer did not appear from nothing. It replaced the encoder-decoder recurrent setup of the 2014 paper “Sequence to Sequence Learning with Neural Networks,” in which an entire sentence was squeezed through one fixed-length vector before being decoded. Attention began life as a **patch** for that bottleneck — letting the decoder look back at every input position instead of trusting a single summary — and by 2017 it had eaten the architecture: “Attention Is All You Need” dropped the recurrence and kept only attention. **Reading 2** is Sutskever’s own retrospective on that 2014 paper, ten years on: **Reading 1 is the architecture; Reading 2 is the bet that scaling it would keep paying.**
- **Pre-training = next-token prediction at scale.** Training simply nudges the weights to predict the next token better across a huge amount of text. Frontier models are this same loop, vastly scaled, plus later instruction- and alignment-tuning stages the lecture doesn’t cover (see Reading 2).

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to get you started; the fuller reasoning follows after all five, so you can settle the argument on your own. Have a go before reading either.

- The model only ever predicts “the next token.” Given that, why can it write a fluent paragraph yet confidently state a false fact? Where does the hallucination come from?

Clue. Ask what the training objective actually rewards — and whether “true” is one of the things it measures at all. Then ask where a fact physically lives in a system that stores only weights, and what happens when the pattern around a fact is strong but the fact itself is thin or absent in the training data.

- Attention lets a token pull information from earlier tokens. How does that help explain why giving a model more relevant context in the prompt (as you did in Week 2’s RAG — Retrieval-Augmented Generation) improves its answer?

Clue. Attention can only reach what is inside the context window. So compare two situations: the answer is somewhere in the model’s training data, versus the answer is sitting in the prompt in front of it. One of those is a recall problem and the other is a lookup problem — and the model is much better at one of them. Which, and why?

- Tokenisation means the model sees tokens, not letters or digits. Name a task in your field where that hidden detail could make a model fail in a surprising way.

Clue. Look for tasks that care about the surface of a string rather than its meaning — digit counts, identifier and code formats, spelling and wordplay, exact arithmetic on long numbers. Ask what the model literally cannot see once the text has been chopped into tokens.

- One attention head can express only one relationship per token. Take a document you actually work with — a contract, a lab report, a long email thread. Find a single word whose meaning depends on two or three different earlier passages at once. What would go wrong if the model tracked only one of them?

Clue. Do it the other way round: pick the word first, then list every earlier passage you had to hold in your head to read it correctly. Count them. That count is roughly how many relationships that single word is standing in at once — and one head can carry one.

- Attention compares every token against every earlier token. If you double the length of your prompt, what happens to the amount of comparison the model must do — and what does that suggest about the habit of pasting an entire document into the chat window?

Clue. If every token compares itself against every earlier token, and you double the number of tokens, how many comparisons is that? Then separate two things that are often confused: what you are billed for, and what the machine actually has to do.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own. If yours differs and you can defend it, bring it.

Q1 — Fluent, and confidently wrong.

Fluency and truth are two different targets, and training only aims at one of them. The objective rewards the **plausible** next token, not the **true** one. The model has learned the shape of correct-sounding text extremely well; facts sit in the weights only as diffuse statistical association. Where the pattern is strong but the fact is thin, absent or contradicted in training data, the most plausible continuation is a confident invention — a citation with a real-looking author, journal and year. Nothing in the loop asks “do I actually know this?” So hallucination is not a defect bolted onto fluency; it is fluency running without a truth constraint. That is why grounding — retrieval, tools, citations, verification — is an engineering fix rather than a prompting trick.

Q2 — Why context in the prompt helps.

Attention can only pull from what is inside the window. Putting the relevant passage in the prompt places a token there whose **key** matches the model’s **query**, so the weight lands on your document rather than on the model’s diffuse pre-training priors. Retrieval-Augmented Generation (RAG) is exactly this move: convert a **recall** problem, which the model is bad at, into a **lookup** problem, which attention is built for. The same mechanism explains the flip side — padding the prompt with irrelevant material spreads the weights thinner and can make answers worse, not better.

Q3 — Where token-blindness bites.

The tell is any task about the **surface form** of a string rather than its meaning: checking whether an identifier has the right number of digits, validating a medical or accounting code, exact character counts, spelling and wordplay, precise arithmetic on long numbers, alignment in a table, transliteration in a minority script. In each case the thing you care about lives **below** the token boundary, where the model has no vision. The professional response is not a cleverer prompt — it is to hand that step to a tool the model calls (a regular expression, a validator, a calculator, a few lines of code) and let the model orchestrate rather than count.

Q4 — One word, several anchors.

A worked instance, if you want one to compare against: in a contract, “such termination” depends at once on which clause defined **termination**, on who the acting party is, and on a governing-law sentence three pages earlier. Your intuition that a word points to several places simultaneously is precisely what multiple heads exist to serve. The practical lesson is about failure mode: when a model gets one of those links wrong, the output stays perfectly

grammatical and confident. It does not look broken. Which is why review of AI output has to be targeted at the linkages — references, antecedents, defined terms — rather than at whether the prose reads well.

Q5 — Long prompts are not free.

Standard attention compares every token with every earlier token, so the comparison work grows roughly with the **square** of the sequence length: double the prompt and you quadruple that work. Providers bill per token linearly and use heavy optimisation (prompt caching, efficient attention kernels, sub-quadratic variants), so you will not see a clean 4× on your invoice — but latency, cost and the risk of the model attending to the wrong thing all climb faster than intuition suggests. The discipline that follows: retrieve the two relevant pages rather than pasting two hundred. “Just give it everything” is a design choice with a price attached, and in Class 4 it becomes a real constraint when agents start looping over long contexts.

SOURCE LINKS

Video · youtube.com/watch?v=kCc8FmEb1nY

Code (nanoGPT) · github.com/karpathy/nanoGPT

Companion · Karpathy, “Intro to Large Language Models” (Class 1 Reading 3)

Companion · **Reading 2**, Sutskever at NeurIPS 2024 — mechanism (this reading) + trajectory (that one).

CAVEATS

1. This summary is not a substitute for the lecture. It is a map, not the territory — watch the source; it is the authority.
2. This summary was generated and fact-checked using Anthropic models against the lecture. Flag anything that looks wrong.